

5.10 Compose 3/3 — Snapshot-состояние, Semantics и тесты UI, Навигация, Adaptive UI, Interop и перф-процесс — учебный конспект (издание «объясняем просто»)

TL;DR

- **Не всё состояние экрана — это MVI-state.** У экрана три уровня состояния: бизнес-данные (во ViewModel/MVI), UI-состояние экрана и состояние отдельных элементов (скролл, фокус, черновик текста) — последние два часто должны жить прямо в композиции через `remember`/`rememberSaveable`. Тащить каждый чих через `Action → Actor → Effect → Reducer` — анти-паттерн «водопровод ради стакана воды».
- **Semantics — один контракт на двоих:** и TalkBack (скринридер для незрячих), и UI-тесты Compose смотрят на одно и то же дерево семантики. Если кнопку не может найти тест — её не может найти и незрячий пользователь. Это делает доступность и тестируемость одной инженерной задачей, а не двумя.
- **Навигация — это состояние, а не магия:** Navigation 3 официально делает back stack (стопку открытых экранов) обычным списком-состоянием, которым владеет приложение, — ровно та философия UDF, на которой построена вся наша MVI-серия. Ваш FragmentAttacher, Nav2 и Nav3 сравниваются по одним и тем же пяти осям.
- **Adaptive UI — не «вторая версия приложения»:** раскладка = функция от *того же* STATE плюс класс размера окна. Отдельный бизнес-state для планшета — ошибка, которая потом мстит при сохранении состояния.
- **Перформанс не меряют «на глаз» и не в debug-сборке:** есть воспроизводимый цикл «гипотеза → трейс → фикс → бенчмарк → регрессионный тест». В конце — errata: 7 точечных исправлений к частям 1/2 и 2/2 (включая баг в нашем же примере с `Channel(DROP_OLDEST)`)).

Как читать этот конспект

Это третья, финальная часть серии (после 5.8 «Rendering, XML, MVI-мотивация» и 5.9 «MVI под микроскопом»). Нумерация тем продолжается: здесь Темы 13–22.

Формат этой части отличается. Первые две написаны «сеньор — сеньору». Эта — в режиме наставника: представьте, что материал объясняют джуну, который умный, но многого ещё не видел. Поэтому: (1) редкие термины и аббревиатуры расшифровываются прямо в тексте при первом упоминании — вот так: *IME (Input Method Editor — по-простому, экранная клавиатура)*; (2) у каждой темы есть блок «Зачем это знать» — почему тема вообще попала в конспект; (3) сложные механики начинаются с бытовой аналогии и только потом идёт точная формулировка; (4) в конце каждой темы — пара вопросов «Проверь себя» с короткими ответами. В самом конце документа — мини-словарь всех терминов части и блок правок к предыдущим частям.

Связь с MVI-частью сохранена: примеры продолжают жить в мире `BenefitsScreen`, `BenefitsState` и `acceptAction` из части 2/2, чтобы вы видели, как новые темы стыкуются с уже разобранный архитектурой.

Key Findings

1. **Compose замечает изменения через snapshot-систему:** во время выполнения `composable`-функции рантайм записывает, какие «наблюдаемые ячейки» (`State`) она прочитала, и при записи в ячейку перерисовывает только тех, кто её читал. `remember` защищает значение от рекомпозиции, `rememberSaveable` — от смерти процесса, новый `retain` — от поворота экрана без сериализации.
 2. **Три уровня состояния экрана** — `business state`, `screen UI state`, `UI element state` — требуют разных владельцев. Официальное правило: поднимать состояние до *наименьшего общего владельца*, а не автоматически до `ViewModel`.
 3. **`TextFieldState` (новый API текстовых полей) решает класс багов «прыгающих букв»:** текст, курсор и `composition` клавиатуры живут в одном синхронном объекте, а в MVI через `snapshotFlow` + `debounce` уходит только осмысленный ввод.
 4. **Дерево семантики ≠ дерево `composable`-функций:** кнопка с иконкой и текстом — один узел (`merged`), и это одновременно то, что «видит» `TalkBack`, и то, по чему ищет `onNodeWithText` в тестах.
 5. **UI-тест Compose закрывает ровно две стрелки MVI-цикла:** «STATE → что видно на экране» и «жест → какой ACTION ушёл наверх». Переходы состояний остаются за табличными тестами `Reducer` из части 2/2 — слои не дублируют друг друга.
 6. **Навигацию надо мысленно разложить на пять частей** (команда, состояние `back stack`, внешний `side effect`, возврат результата, восстановление после смерти процесса) — тогда сравнение `FragmentAttacher` / `Navigation 2` / `Navigation 3` / координатор становится инженерным выбором, а не религией.
 7. **Window size class, а не «телефон или планшет»:** решения о раскладке принимаются по классу ширины окна, потому что «планшет» может показать ваше приложение в узком окне `split-screen` (разделённый экран), а телефон — в ландшафте на полке.
 8. **Interop с View — это про владение жизненным циклом:** `AndroidView` имеет `update`/`onReset`/`onRelease`, а `WebView`/`MapView` требуют явного проброса `lifecycle`. «Вставил и работает» без очистки — это утечки памяти, которые видно только в проде.
 9. **Baseline Profile обязан быть проверен, а не «включён»:** `Macrobenchmark` с `CompilationMode.None` против `CompilationMode.Partial` — единственный честный способ узнать, что профиль действительно ускорил ваши экраны.
-

Details

Тема 13. Snapshot-состояние: как Compose вообще замечает изменения

Зачем это знать. Это фундамент всего Compose. Пока вы не понимаете, *как* Compose узнаёт об изменениях, `remember`, `derivedStateOf` и «почему список не обновился» остаются магией и источником самых частых багов новичка. А ещё это фундамент Темы 14 — без него невозможно осознанно решить, что класть в MVI, а что нет.

Аналогия. Excel-таблица. Ячейки — это состояние (`mutableStateOf`), формулы — composable-функции. Когда вы меняете ячейку B2, Excel не пересчитывает весь лист: он знает, какие формулы ссылаются на B2, и пересчитывает только их. Compose делает то же самое: во время выполнения composable он *записывает*, какие ячейки та прочитала, и при изменении ячейки *инвалидирует* (инвалидация — пометка «это место устарело, перерисуй») только читателей.

13.1. `mutableStateOf` — наблюдаемая ячейка

kotlin

`@Composable`

`fun Counter() {`

`// ПЛОХО: без remember объект состояния пересоздаётся при КАЖДОЙ рекомпозиции`
`// Нажали кнопку → count стал 1 → рекомпозиция → созданся НОВЫЙ mutableStateOf`
`// val count = mutableStateOf(0)`

`// ХОРОШО: remember говорит «сохрани этот объект между переывзовами функции»`
`var count by remember { mutableStateOf(0) }`

`Button(onClick = { count++ }) {`
 `Text("Нажато: $count")`
`}`

`}`

Точная формулировка: `mutableStateOf` создаёт объект `MutableState` — «ячейку», чтение которой Compose отслеживает во всех трёх фазах (`Composition`, `Layout`, `Drawing` — см. часть 1/2). Запись в ячейку инвалидирует только те области (scopes), которые её читали. Обычная `var count = 0` невидима для Compose: её изменение никого не перерисует.

Важная тонкость с коллекциями. `mutableStateOf(mutableListOf<Item>())` — ловушка: Compose следит за *заменой значения ячейки*, а не за мутациями внутри списка. `list.add(item)` ничего не перерисует, потому что ссылка в ячейке не поменялась. Два правильных пути: либо неизменяемые снимки (`state = state + item` — новый список в ячейку), либо наблюдаемая коллекция `mutableStateListOf<Item>()`, у которой отслеживается каждая операция. Для MVI-мира из части 2/2 актуален первый путь: `STATE` — `immutable`, Reducer возвращает новый объект.

13.2. Snapshot — почему это безопасно и согласованно

«Snapshot» переводится как «снимок». Система снапшотов Compose работает как мини-транзакции в базе данных (транзакция — «все изменения применяются разом или никак»): каждая рекомпозиция видит *согласованный снимок* всех ячеек на один момент времени — вы никогда не увидите «наполовину обновлённую» пару значений внутри одного кадра. Писать в state можно из любого потока: Compose сам синхронизирует изменения через снапшоты. Для джуна практический вывод один: **не нужно самому городить синхронизацию вокруг `MutableState`** — но нужно, чтобы связанные поля менялись вместе (что в MVI и так гарантирует Reducer, — вот и первая стыковка с частью 2/2).

13.3. Три уровня «выживаемости»: `remember`, `rememberSaveable`, `retain`

API	Переживает рекомпозицию	Переживает поворот экрана	Переживает process death	Как хранит
<code>remember { }</code>	✓	✗	✗	объект в памяти композиции
<code>retain { }</code> (новый, Compose 1.10)	✓	✓	✗	объект в памяти вне композиции (без сериализации)
<code>rememberSaveable { }</code>	✓	✓	✓	сериализует в <code>Bundle</code>
<code>ViewModel (+ SavedStateHandle)</code>	✓	✓	✓ (только handle)	объект в памяти + Bundle для handle

Расшифровки. **Process death** — система убила процесс приложения в фоне ради памяти; при возврате пользователя Activity пересоздаётся «с нуля», но система возвращает сохранённый `Bundle`. **Bundle** — маленький контейнер «ключ → примитив/Parcelable», который система умеет сохранять; он передаётся через межпроцессную границу с лимитом порядка 1 МБ на транзакцию — поэтому туда кладут ID и введённый текст, а не списки товаров. **Сериализация** — превращение объекта в байты для сохранения; `retain` тем и ценен, что держит «несериализуемое» (например, тяжёлый плеер или подключение) живым через поворот. `retain` — свежий API: проверяйте статус в актуальной документации, прежде чем тащить в прод.

`rememberSaveable` для своих типов требует `Saver` — инструкцию «как разобрать объект на примитивы и собрать обратно». У платформенных состояний (`rememberLazyListState`, `rememberTextFieldState`) `Saver` уже встроен.

13.4. `derivedStateOf` — кэш с фильтром «а результат-то изменился?»

```
kotlin

val listState = rememberLazyListState()

// firstVisibleItemIndex меняется на КАЖДЫЙ чих скролла.
// Кнопке же важно только «мы вверху или нет» (true/false).
// derivedStateOf пересобирает читателей ТОЛЬКО когда меняется РЕЗУЛЬТАТ вычисления
val showScrollToTop by remember {
    derivedStateOf { listState.firstVisibleItemIndex > 0 }
}

if (showScrollToTop) {
    ScrollToTopButton(/* ... */)
}
```

Правило выбора: `derivedStateOf` нужен, когда **вход меняется часто, а выход — редко**. Классика: индекс скролла → булево «показать кнопку». Анти-паттерн новичка: заворачивать в `derivedStateOf` простое склеивание двух полей (`"$name $surname"`) — тут вход и выход меняются одинаково часто, вы добавили объект и косвенность, а сэкономили ноль. И не забывайте `remember` вокруг — иначе `derived`-объект пересоздаётся каждый раз, и весь смысл теряется.

13.5. `snapshotFlow` — мост из мира ячеек в мир Flow

`snapshotFlow { ... }` наблюдает за чтением ячеек внутри блока и превращает изменения в холодный `Flow` (эмитит при изменении результата). Это официальный мост «Compose-state → корутинный мир» — и главный канал, по которому локальное UI-состояние сообщает что-то в MVI:

kotlin

```
// Скролл-позиция — локальное UI-состояние. Но аналитике/MVI интересен факт «до  
LaunchedEffect(listState) {  
    snapshotFlow { listState.firstVisibleItemIndex }  
        .distinctUntilChanged()  
        .collect { index -> viewModel.acceptAction(BenefitsAction.ListScrolled(  
    })
```

Обратный мост — `collectAsStateWithLifecycle` из части 2/2: Flow → ячейка. Пара «`snapshotFlow / collectAsStateWithLifecycle`» — это две двери между двумя мирами, и обе вы теперь знаете.

Проверь себя.

- Почему `val x = mutableStateOf(0)` без `remember` внутри `composable` — баг? — Объект пересоздаётся при каждой рекомпозиции, значение вечно сбрасывается в начальное.
- Почему `list.add()` внутри `mutableStateOf(mutableListOf())` не обновляет UI? — Compose следит за заменой значения ячейки, а не за мутациями внутри объекта; нужен новый список или `mutableStateListOf`.
- Чем `retain` отличается от `rememberSaveable`? — `retain` держит объект в памяти через поворот без сериализации, но не переживает `process death`; `rememberSaveable` сериализует в Bundle и переживает и то и другое.

Тема 14. Что НЕ надо тащить в MVI: три уровня состояния экрана

Зачем это знать. Часть 2/2 могла оставить впечатление «любое состояние — через Action → Actor → Effect → Reducer». Это не так, и это самая частая ошибка команд, внедривших MVI: экран на 500 строк, где раскрытие аккордеона требует три новых класса и правку Reducer'a. Умение провести границу — признак зрелости, и об этом прямо спрашивают на собеседованиях («что вы храните во ViewModel, а что нет?»).

Аналогия. Паспорт и стикеры на мониторе. Паспорт (бизнес-данные) хранится в сейфе, выдаётся по процедуре, его потеря — катастрофа. Стикер «позвонить Маше» (позиция скролла, раскрыт ли блок) живёт на мониторе: потерялся при переезде — ну и ладно, напишем новый. Если каждый стикер оформлять через сейф и журнал выдачи — работа встанет.

14.1. Классификация: кто где живёт

Уровень	Что это	Примеры	Владелец
Business state	Данные предметной области и их загрузка	список бенефитов, профиль, ошибка сети, выбранный тариф	ViewModel / MVI STATE
Screen UI state	Состояние экрана, о котором должна знать логика	выбранный фильтр, подтверждённый поисковый запрос, режим редактирования	чаще MVI STATE (особенно если нужно после process death)
UI element state	Механика конкретных виджетов	позиция скролла, раскрыт ли аккордеон, черновик текста до отправки, фокус, прогресс анимации	композиция: remember / rememberSaveable / state-объекты платформы

Официальное правило Android называется **state hoisting** (хостинг состояния — «поднятие» его вверх по дереву): поднимайте состояние до *наименьшего общего владельца* — самого низкого места в дереве, откуда его видят все, кому оно нужно. Не «всегда во ViewModel», а «настолько низко, насколько можно; настолько высоко, насколько нужно».

14.2. Почему «всё в MVI» — это дорого

Раскрытие аккордеона через полный цикл MVI стоит: новый **ACTION**, новый **EFFECT**, ветка в Reducer, строка в табличных тестах — четыре правки ради флажка **Boolean**, который никому кроме этого виджета не нужен. Хуже того: табличные тесты Reducer из части 2/2 начинают тонуть в шуме из «UI-чихов», и главная ценность — читаемая спецификация *бизнес-переходов* — размывается. Локальный вариант — одна строка:

```
kotlin
```

```
var expanded by rememberSaveable { mutableStateOf(false) } // переживёт и повор
```

14.3. Готовые держатели состояния элементов

У платформы уже есть классы-держатели для типовой механики — их не надо изобретать и не надо переносить во ViewModel (они держат ссылки на UI и живут по lifecycle композиции):

- `rememberLazyListState()` — позиция списка (со встроенным Saver: переживает пересоздание);
- `rememberPagerState()` — текущая страница пейджера;
- `remember { SnackbarHostState() }` — очередь снэкбаров (в связке с news-паттерном из Темы 11 части 2/2);
- `remember { FocusRequester() }` — программное управление фокусом;
- `remember { Animatable(0f) }` — прерываемые анимации (Тема 22).

Если у экрана несколько связанных кусочков `element-state` (например, форма: фокус + валидация подсветки + раскрытые подсказки) — сгруппируйте их в **plain state holder** (обычный класс-держатель, без `ViewModel`):

kotlin

```
/** Держатель локального UI-состояния формы: живёт в композиции, в MVI не ходит
@Stable
class BenefitsFormUiState(
    val listState: LazyListState,
    val snackbarHostState: SnackbarHostState,
) {
    var expandedSectionId by mutableStateOf<String?>(null)
}

@Composable
fun rememberBenefitsFormUiState(
    listState: LazyListState = rememberLazyListState(),
    snackbarHostState: SnackbarHostState = remember { SnackbarHostState() },
): BenefitsFormUiState = remember(listState, snackbarHostState) {
    BenefitsFormUiState(listState, snackbarHostState)
}
```

14.4. Четыре вопроса-фильтра

Прежде чем нести состояние во `ViewModel`, задайте четыре вопроса. Если на все ответ «нет» — оставляйте в композиции:

1. Нужно ли оно **после process death**? (черновик заявления — да; прогресс ripple-анимации — нет)
2. Нужно ли оно **бизнес-логике или аналитике**? (выбранный фильтр — да; раскрытый аккордеон — обычно нет)
3. Влияет ли оно на **другие экраны или запросы**? (query — да; позиция скrolла — нет)
4. Хотите ли вы видеть его в **табличных тестах Reducer**? (если мысль «это будет шум» — значит, ему не место в `STATE`)

Пограничный случай — «нужно после process death, но это element-state» (позиция скролла в ленте новостей): решается `rememberSaveable`/встроенным Saver'ом, во ViewModel по-прежнему не надо.

Проверь себя.

- *Раскрыт ли фильтр-панель — какой это уровень? — UI element state:* `rememberSaveable` в композиции, если только раскрытие не меняет запросы к серверу.
- *Почему LazyListState не хранят во ViewModel?* — Он держит ссылки на измерения конкретной композиции и живёт её lifecycle'ом; ViewModel живёт дольше view — получите утечку и бессмысленный объект.
- *Что такое «наименьший общий владелец»?* — Самая низкая точка дерева, из которой состояние видно всем его читателям и писателям.

Тема 15. Текстовый ввод по-новому: `TextFieldState` вместо `value/onValueChange`

Зачем это знать. Текстовые поля — место, где джуна поджидает самый загадочный баг Compose: «буквы прыгают и дублируются при быстрой печати». Причина — не мистика, а асинхронный круг данных в старом API. Новый state-based API закрывает класс проблем целиком, и Google официально рекомендует мигрировать на него.

Аналогия старой проблемы. Вы диктуете секретарю текст по телефону, а он после каждой буквы зачитывает вам весь документ обратно, и вы продолжаете печатать с *его версии*. Пока связь мгновенная — всё хорошо. Как только появляется задержка (а у нас между полем и истиной лежит ViewModel, иногда с фильтрацией) — вы печатаете поверх устаревшей версии: буквы теряются, курсор скачет.

15.1. Старый API и его врождённая гонка

```
kotlin
```

```
// Старый подход: значение живёт где-то снаружи и возвращается через колбэк
TextField(
    value = state.query, // истина приходит СВЕРХУ
    onChange = { viewModel.acceptAction(QueryChanged(it)) }, // изменение
)
```

Каждое нажатие — круг: поле → колбэк → ViewModel → Reducer → StateFlow → рекомпозиция → поле. Если круг занял больше одного кадра (а в нашей архитектуре из части 2/2 между полем и полем лежит ещё Rx-мост!), IME (Input Method Editor — экранная клавиатура) успевает прислать следующий символ *по старому состоянию*. Отсюда дубли, скачки курсора и слом composition — «подчёркнутого» текста, который клавиатура ещё дособирует (актуально для японского/китайского ввода и свайп-набора).

15.2. Новый API: состояние живёт в самом поле

kotlin

```
val queryState = rememberTextFieldState() // текст + курсор + composition в ОДН

BasicTextField(
    state = queryState, // поле читает и пишет синхронно, без круга через ViewM
)
```

`TextFieldState` — снапшот-объект (Тема 13), в котором синхронно живут текст, выделение (selection) и composition. Ввод применяется мгновенно и без гонок; у `rememberTextFieldState` встроен Saver — черновик переживает поворот и process death.

15.3. Фильтрация и маски: `InputTransformation` и `OutputTransformation`

kotlin

```
/** Пускаем только цифры: неподходящее изменение просто откатываем */
object DigitsOnlyTransformation : InputTransformation {
    override fun TextFieldBuffer.transformInput() {
        if (!asCharSequence().all { it.isDigit() }) revertAllChanges()
    }
}

BasicTextField(
    state = rememberTextFieldState(),
    inputTransformation = InputTransformation.maxLength(16).then(DigitsOnlyTransformation),
    outputTransformation = CardNumberSpaces, // визуальные пробелы, см. ниже
)

/** Показ «1234 5678 ...» — только ВИЗУАЛЬНО: реальное значение остаётся без пробелов */
object CardNumberSpaces : OutputTransformation {
    override fun TextFieldBuffer.transformOutput() {
        var i = 4
        while (i < length) {
            insert(i, " ")
            i += 5
        }
    }
}
```

Разница принципиальна: `InputTransformation` меняет, **что сохраняется** (фильтр/лимит), `OutputTransformation` — только **что рисуется** (маска карты, телефона). В старом API визуальные маски были минным полем `VisualTransformation` с ручным маппингом позиций курсора — теперь буфер делает это за вас.

15.4. Стыковка с MVI: черновик — локально, смысл — наверх

Правило из Темы 14 в действии: черновик текста — `element-state`, подтверждённый запрос — `screen/business state`:

kotlin

`@Composable`

```
internal fun BenefitsSearchBar(onQueryConfirmed: (String) -> Unit) {
    val queryState = rememberTextFieldState()

    // Мост «ячейки → Flow» из Темы 13: наверх уходит не каждая буква,
    // а устоявшийся ввод (debounce = «подождать паузу в печати», требует @OptI
    LaunchedEffect(queryState) {
        snapshotFlow { queryState.text.toString() }
            .debounce(300)
            .distinctUntilChanged()
            .collect(onQueryConfirmed) // → viewModel.acceptAction(QueryChanged

    }

    BasicTextField(state = queryState /* ... */)
}
```

Так MVI-цикл получает только события, имеющие бизнес-смысл, Reducer-тесты не тонут в посимвольном шуме, а поле остаётся мгновенно отзывчивым. И заметьте: `requestId-guard` из Темы 6 части 2/2 здесь всё ещё нужен — быстрые смены `query` порождают те же `stale`-результаты поиска.

Оговорка про версии: `state-based BasicTextField` стабилен в `foundation`; у Material 3 `TextField` `state`-вариант появлялся позже и от версии к версии менялся — прежде чем использовать в проде, сверьте версию `material3` в вашем `libs.versions.toml`.

Проверь себя.

- Откуда берутся «прыгающие буквы» в старом API? — Асинхронный круг поле → ViewModel → поле: быстрый ввод накладывается на устаревшее значение.
- Чем `InputTransformation` отличается от `OutputTransformation`? — Первая меняет сохраняемое значение (фильтр), вторая — только отображение (маска).
- Каждую ли букву отправлять в `acceptAction`? — Нет: черновик локален, в MVI через `snapshotFlow` + `debounce` уходит осмысленный ввод.

Тема 16. Semantics: один контракт для доступности и тестов

Зачем это знать. В Compose есть красивое инженерное совпадение, о котором джуны часто не подозревают: *доступность* (accessibility — возможность пользоваться приложением людям с ограничениями: незрячим, слабовидящим, людям с моторными нарушениями) и *UI-тесты* работают через один и тот же механизм — дерево семантики. Сделали компонент доступным — он автоматически стал тестируемым, и наоборот. Это превращает «доступность» из «задачи на потом» в бесплатный побочный эффект хорошей инженерии.

Аналогия. Дом с табличками. Зрячий гость ориентируется по виду комнат, но пожарный в дыму (TalkBack) и робот-инспектор (тест) читают таблички на дверях: «Кухня», «Выход», «Кладовая, закрыто». Если табличек нет — для них дома просто не существует, каким бы красивым он ни был. Semantics — это и есть таблички.

16.1. Что такое дерево семантики

Параллельно дереву UI Compose строит **semantics tree** — упрощённое машинно-читаемое описание экрана: «это кнопка с текстом "Повторить", включена», «это переключатель, состояние: выключен». Его читают: **TalkBack** (встроенный скринридер Android — озвучивает экран и позволяет управлять жестами), **Switch Access** (управление одной-двумя кнопками для людей с моторными ограничениями), клавиатура/D-pad, автозаполнение — и `ComposeTestRule` в ваших тестах. Дерево семантики ≠ дерево ваших composable-функций: рантайм его сжимает и объединяет.

16.2. Merged и unmerged: почему кнопка — один узел

Кнопка внутри состоит из поверхности, ряда, иконки и текста, но для пользователя TalkBack это *одно* действие. Поэтому узлы с `mergeDescendants = true` (его ставят кликабельные модификаторы автоматически) **сливают** потомков в один семантический узел — это *merged tree*, дерево по умолчанию и для доступности, и для тестов. Отсюда два практических следствия:

- `onNodeWithText("Повторить")` найдёт *кнопку целиком*, и `performClick()` по ней — то, что надо;
- а вот найти *иконку внутри кнопки* в merged-дереве нельзя — она «растворилась». Для таких проверок тесту передают `useUnmergedTree = true`.

Крайний случай слияния — `Modifier.clearAndSetSemantics { ... }`: «сотри всю внутреннюю семантику и опиши узел заново». Нужен для сложных кастомных компонентов, где автоматическое описание получается мусорным («кнопка, картинка, текст, текст, картинка» вместо «Карта Gold, баланс 12 000»).

16.3. Минимальный набор свойств, который должен знать каждый

kotlin

```
// 1) Иконка без текста ОБЯЗАНА иметь описание (иначе TalkBack скажет «без метки»)
// null допустим только для чистой декорации.
Icon(
    imageVector = Icons.Filled.Favorite,
    contentDescription = "Добавить в избранное",
)

// 2) Кастомный переключатель: role объясняет «что это», stateDescription — «в каком состоянии»
Row(
    modifier = Modifier
        .toggleable(value = isEnabled, role = Role.Switch, onValueChange = onToggle)
        .semantics { stateDescription = if (isEnabled) "Включено" else "Выключено" }
) { /* ... */ }

// 3) Заголовок секции: TalkBack позволит прыгать по заголовкам, как по оглавлению
Text(
    text = "Ваши бенефиты",
    modifier = Modifier.semantics { heading() },
)
```

Ещё два правила гигиены: **размер цели касания** — интерактивные элементы должны иметь зону нажатия минимум 48×48 dp (Material-компоненты обеспечивают это сами через `minimum interactive size` — не отключайте бездумно); **порядок обхода** — TalkBack идёт по экрану в порядке, который обычно совпадает с визуальным, но для хитрых раскладок его правят (`isTraversalGroup`)/(`traversalIndex`).

16.4. `testTag` — честный костыль

`Modifier.testTag("benefits_list")` вешает на узел метку только для тестов. Правило хорошего тона: сначала пытайтесь находить узлы так, как их находит пользователь — по тексту и `contentDescription` (это заодно проверяет, что семантика вообще есть!). `testTag` — для неоднозначных случаев: третий элемент в списке одинаковых карточек, контейнер без текста.

Проверь себя.

- Почему тест не находит иконку внутри кнопки? — Кнопка сливает потомков (merged tree); нужен `useUnmergedTree = true` или поиск по самой кнопке.
- Чем `contentDescription` отличается от `stateDescription`? — Первое — «что это» («Добавить в избранное»), второе — «в каком оно состоянии» («Включено»).
- Что даёт `Modifier.semantics { heading() }`? — TalkBack позволяет навигировать по заголовкам, как по оглавлению документа.

Тема 17. UI-тесты Compose: `ComposeTestRule` без страха

Зачем это знать. В части 2/2 мы построили пирамиду: 70% Reducer, 20% Actor, 10% smoke. Но одна стрелка осталась непокрытой: «*STATE* → что реально видит пользователь» и «*жест пользователя* → какой *ACTION* ушёл». Именно эти две стрелки закрывают UI-тесты — и благодаря stateless-паттерну из Темы 11 (экран = функция от `state` и `onAction`) они пишутся неприлично просто: не нужны ни моки, ни ViewModel, ни DI.

17.1. Каркас: rule, finders, assertions, actions

```
kotlin

class BenefitsScreenTest {

    @get:Rule
    val composeRule = createComposeRule() // поднимает пустую Activity под капотом

    @Test
    fun errorState_showsRetry_andClickSendsAction() {
        val actions = mutableListOf<BenefitsAction>() // «ловушка» для действий

        composeRule.setContent {
            BenefitsScreen(
                state = BenefitsState(error = BenefitsError.Network), // подаём ошибку
                onAction = actions::add,
            )
        }

        composeRule.onNodeWithText("Повторить") // finder: найти узел в дереве
            .assertIsDisplayed() // assertion: проверить свойство
            .performClick() // action: выполнить жест

        assertEquals(listOf(BenefitsAction.RetryClicked), actions) // жест превратился в действие
    }
}
```

Запомните тройку глаголов API: **найти** (`onNodeWithText`, `onNodeWithContentDescription`, `onNodeWithTag`, `onAllNodesWithText`), **проверить** (`assertIsDisplayed`, `assertIsEnabled`, `assertTextEquals`, `assertCountEquals`), **сделать** (`performClick`, `performTextInput`, `performScrollTo`, `performTouchInput { swipeLeft() }`). Всё это работает по дереву семантики из Темы 16 — вот и обещанная связка «доступность = тестируемость».

`createComposeRule` хватает для stateless-экранов;

`createAndroidComposeRule<MainActivity>()` нужен, когда тесту требуется настоящая Activity и её ресурсы (гибридные экраны, темы из XML).

17.2. Синхронизация: почему тесты «сами ждут» и когда ждать вручную

Compose-тесты автоматически ждут **idle** (айдл — состояние «всё устаканилось»: нет незавершённой рекомпозиции, layout'a, отрисовки и активных анимаций) перед каждым finder/assertion. Поэтому в 90% случаев никаких `Thread.sleep` — это главное отличие от боли Espresso-эпохи. Ручное ожидание нужно для асинхронщины вне Compose:

```
composeRule.waitForIdle(timeoutMillis = 2_000) { условие }
```

Анимации тестируются управлением часами:

kotlin

```
composeRule.mainClock.autoAdvance = false // забираем время под свой контроль
composeRule.onNodeWithText("Показать").performClick()
composeRule.mainClock.advanceTimeBy(150) // проматываем ровно 150 мс анимации
composeRule.onNodeWithTag("banner").assertIsDisplayed()
```

17.3. Гибридные экраны и автоматические проверки доступности

Если на экране Compose живёт рядом с View (наш Fragment-мир из части 2/2 — ровно такой), в одном тесте спокойно сочетаются `composeRule.onNodeWithText(...)` для Compose-части и `Espresso.onView(withId(...))` для View-части — правила не конфликтуют.

И бесплатный бонус свежих версий `compose-ui-test`:

`composeRule.enableAccessibilityChecks()` — при каждом действии тест автоматически прогоняет проверки доступности (маленькие цели касания, отсутствие описаний, низкий контраст) и валит тест с объяснением. Один вызов — и ваши UI-тесты заодно сторожат доступность. Для ручной проверки живого приложения есть приложение Accessibility Scanner от Google.

17.4. Куда это ставить в пирамиду

UI-тесты — инструментальные (нужен эмулятор/устройство), поэтому они дороже JVM-тестов. Держите их сфокусированными: по одному тесту на каждый важный визуальный вариант `STATE` (loading/content/error/empty) и на каждый жест→ACTION. Переходы между состояниями сюда не тащите — они уже доказаны табличными тестами Reducer. Для защиты от визуальных регрессий («съехала вёрстка») существует отдельный класс инструментов — скриншот-тесты (Compose Preview Screenshot Testing) — знать о них достаточно на уровне «есть такой инструмент».

Проверь себя.

- Почему для теста `BenefitsScreen` не нужен ни `MockK`, ни `ViewModel`? — Экран `stateless: STATE` подаётся параметром, `ACTION` ловится в список — чистая функция «данные → семантика».
 - Что такое `idle` и почему тесты не требуют `sleep`? — Тест сам ждёт завершения рекомпозиции/анимаций перед каждой операцией.
 - Как протестировать середину анимации? — `mainClock.autoAdvance = false` + `advanceTimeBy(...)`.
-

Тема 18. Навигация как state machine: от `FragmentAttacher` до `Navigation 3`

Зачем это знать. В части 2/2 навигация появлялась как `NEWS` и паттерн `pendingNavigation + acknowledgement` — это был взгляд «изнутри одной фичи». Для целостной картины (и для собеседования) навигацию надо увидеть как отдельную state machine со своими компонентами. Тогда «какую библиотеку взять» превращается из религиозного спора в таблицу трейд-оффов.

Аналогия. Навигация — это не «телепорт», а стопка тарелок. **Back stack** (стек возврата) — стопка открытых экранов: новый экран кладётся сверху, кнопка «назад» снимает верхний. Пока вы думаете о навигации как о «команде телепортироваться», вы забываете, что у стопки есть *состояние*, которое надо хранить, восстанавливать и синхронизировать.

18.1. Пять частей любой навигации

Разложите любой переход на пять независимых вопросов — и сравнение библиотек станет механическим:

1. **Команда** — намерение перейти («открой детали бенефита X»). У нас это `NEWS` или `pendingNavigation` из части 2/2.
2. **Состояние** — сам back stack: какие экраны открыты и в каком порядке. Кто им владеет — вы или фреймворк?
3. **Внешний side effect** — запуск чужого Activity, браузера, системного диалога разрешений. Это не навигация внутри приложения, и смешивать их — ошибка: для внешнего есть Activity Result API.
4. **Возврат результата** — экран Б должен вернуть выбранное значение экрану А.
5. **Восстановление** — что случится со стопкой после process death (смерть процесса, см. Тему 13)?

18.2. Четыре подхода на одной таблице

Ось	FragmentAttacher (ваш проект)	Navigation 2 (NavController)	Navigation 3	Свой координатор
Кто владеет back stack	FragmentManager (система)	NavController (библиотека)	вы: стек — обычный список-состояние	вы, но всё пишете сами
Маршруты	Fragment + строковый тег	типизированные (<code>@Serializable</code>) маршруты (с 2.8)	(<code>NavKey</code>)-объекты (<code>@Serializable</code>)	как решите
Переживает process death	✓ FragmentManager сохраняет стек сам	✓ сохраняет автоматически	✓ через (<code>rememberNavController</code>) (ключи сериализуемы)	только если сами написали
Predictive back	только если явно поддержали в атачере	✓ поддержан	✓ поддержан из коробки	сами
Compose- нативность	нет: Compose живёт внутри фрагментов	середина: NavHost в Compose, идеология своя	да: спроектирован под Compose	зависит
Когда выбирать	большая живая коддовая база на фрагментах (ваш случай)	зрелый мейнстрим для Compose- приложений	новые Compose-first приложения; статус API проверить (активно развивается)	особые требования: сложные стеки виджетов, A/ B-навигация

Расшифровка: deep link — ссылка (из пуша, браузера, другого приложения), открывающая конкретный экран внутри приложения; поддерживается всеми четырьмя подходами, но в самописных решениях её стоимость самая высокая — восстановление «синтетического» стека под экраном руками.

18.3. Navigation 3: навигация — это буквально состояние

Идеологический прорыв Nav3 в одном предложении: **back stack — это просто `SnapshotStateList`, которым владеете вы, а `NavDisplay` его отображает**. Никакого скрытого состояния внутри контроллера — та самая UDF-философия «UI = f(state)», на которой построена вся наша серия:

kotlin

```
@Serializable data object Home : NavKey
@Serializable data class Details(val id: String) : NavKey

@Composable
fun AppNavigation() {
    // Стек — обычное сохраняемое состояние. Push = add, back = removeLast. Всё
    val backStack = rememberNavBackStack(Home)

    NavDisplay(
        backStack = backStack,
        onBack = { backStack.removeLastOrNull() },
        entryProvider = entryProvider {
            entry<Home> { HomeScreen(onOpenDetails = { id -> backStack.add(Deta
            entry<Details> { key -> DetailsScreen(id = key.id) }
        },
    )
}
```

Красивое следствие для нашей MVI-темы: раз стек — состояние, то «навигация как одноразовое событие» частично растворяется — переход становится *изменением состояния*, доступным для тестов и восстановления. Паттерн `pendingNavigation + ack` из части 2/2 остаётся нужен там, где команда пересекает границы владения: внешние `intent`'ы и переходы, инициируемые слоем, не владеющим стеком. (API Nav3 на момент написания активно развивается — сверьте статус перед продом.)

18.4. Predictive back, результаты и восстановление

Predictive back (предиктивный жест «назад») — системная механика Android 14+: пользователь тянет жест назад и *до отпускания* видит анимацию-предпросмотр, куда он вернётся. Требуется `android:enableOnBackPressedCallback="true"` в манифесте и поддержки на уровне навигации. Nav2/Nav3 поддерживают; в кастомном атачере с централизованным `handleBackNavigation()` предпросмотр работает только если вы явно интегрировали новые колбэки — честный ответ для вашего проекта: «поддержка ограничена, проверяем на устройстве».

Возврат результата. В Fragment-мире — `setFragmentManagerResultListener` (штатный API `FragmentManager`, работает с вашим атачером); в Nav2 — `previousBackStackEntry.savedStateHandle`; в Nav3 результат — просто общее состояние (например, во `ViewModel` родителя), потому что стеком владеете вы. Для *внешних* результатов (камера, разрешения, выбор файла) — всегда `ActivityResult API`, независимо от навигационной библиотеки.

Восстановление. Неожиданный плюс «легаси»: `FragmentManager` сохраняет стек фрагментов при `process death` сам — ваш `FragmentAttacher` получает это бесплатно. В `Nav3` за это отвечает `rememberNavBackStack` + сериализуемые ключи: положите в ключ несериализуемое — получите молчаливую потерю стека.

Проверь себя.

- *Чем навигационная команда отличается от навигационного состояния?* — Команда — намерение перейти (одноразовое), состояние — текущая стопка экранов (живёт и восстанавливается).
 - *Почему открытие камеры — не навигация?* — Это внешний `side effect` с результатом: `ActivityResult API`, а не `push` в стек.
 - *Что делает `Nav3` идеологически близким `MVI`?* — `Back stack` — обычное состояние приложения: `UI = f(state)` распространяется и на навигацию.
-

Тема 19. Adaptive UI и edge-to-edge: один STATE — разные раскладки

Зачем это знать. С 2024–2026 Google методично закрывает эпоху «приложение = телефон в портрете»: `edge-to-edge` стал принудительным, большие экраны перестали уважать запрет поворота, появились стабильные `adaptive`-компоненты. Для `Senior`-собеседования это уже гигиенический минимум. А для нашей серии тут есть красивый архитектурный принцип, связывающий всё с `MVI`.

Аналогия. Вода и сосуды. Ваши данные (`STATE`) — вода; телефон, планшет, раскрытый складной — сосуды разной формы. Наливать нужно *одну и ту же воду*, меняя только форму. Команды, которые заводят «отдельную воду для планшета» (второй `business-state`), потом месяцами синхронизируют два стакана.

19.1. Window size class: спрашиваем про окно, а не про устройство

Window size class — официальная классификация *ширины окна* (не устройства!):

Compact (< 600 dp — телефон в портрете), **Medium** (600–840 dp — маленький планшет, раскрытый складной), **Expanded** (≥ 840 dp — планшет, десктоп); в свежих версиях добавились **Large** (≥ 1200 dp) и **Extra-Large** (≥ 1600 dp) для десктопов и внешних мониторов. Почему именно окно: планшет может показать вас в узкой колонке `split-screen` (режим разделённого экрана), а телефон в ландшафте — шире «планшетного» порога. Проверка `if (isTablet)` — анти-паттерн, который эти режимы ломают.

```
kotlin
```

```
@Composable
```

```
internal fun BenefitsAdaptiveScreen(state: BenefitsState, onAction: (BenefitsAc  
    val widthClass = currentWindowAdaptiveInfo().windowSizeClass.windowWidthSiz  
  
    when (widthClass) {  
        WindowWidthSizeClass.COMPACT ->  
            BenefitsSinglePane(state, onAction)           // телефон: список, де  
        else ->  
            BenefitsListDetail(state, onAction)           // ТОТ ЖЕ state: списо  
    }  
}
```

(Имена в API от версии к версии слегка меняются — смысл один: ширина окна попадает в класс.)

19.2. Главный принцип: раскладка = f(тот же STATE, конфигурация окна)

Никакого `TabletBenefitsState`. В едином `BenefitsState` живёт `selectedBenefitId` — а дальше *раскладка сама решает*, что это значит: на Compact выбор открывает экран деталей (push в стек), на Expanded — заполняет правую панель. Бонусы, которые вы получаете бесплатно: переход телефон↔планшетный режим (докинг складного, split-screen) **не теряет выбор**, потому что состояние одно; Reducer-тесты из части 2/2 покрывают обе раскладки сразу; аналитика видит одну модель. Готовые каркасы, чтобы не собирать руками: `NavigationSuiteScaffold` (сам превращает нижнюю навигацию в боковой rail на широких окнах) и `ListDetailPaneScaffold` из стабильной библиотеки Material 3 Adaptive.

Fold posture (поза складного устройства — раскрыт, «книжка», «палатка»/tabletop) — следующий уровень адаптивности: доступна через `FoldingFeature`; для конспекта достаточно знать, что «палатка» = контент сверху, управление снизу (видеозвонки, плееры).

19.3. Edge-to-edge и insets: рисуем под панелями, но уважаем их

Edge-to-edge — приложение рисует от края до края экрана, *под* полупрозрачными системными панелями. С targetSdk 35 (Android 15) это поведение принудительное — «выключить» больше нельзя, можно только правильно приготовить. Ключевое слово — **insets** (отступы-«зоны, занятые системой»): статус-бар сверху, жестовая панель снизу, **cutout** (вырез под камеру), и — внезапно для новичков — экранная клавиатура (IME) тоже inset.

Рецепт из трёх правил:

kotlin

```
// 1) Activity: включаем edge-to-edge (до targetSdk 35 — явно; после — оно и так)
enableEdgeToEdge()

// 2) Фоны и контейнеры НЕ отступают — пусть красиво уходят под панели.
// Контент — отступает. Scaffold делает это за вас через innerPadding:
Scaffold { innerPadding ->
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        contentPadding = innerPadding, // контент не залезет под статус-бар и ж
    ) { /* ... */ }
}

// 3) Экран с полем ввода: приподнимаемся над клавиатурой
Column(modifier = Modifier.imePadding()) { /* форма */ }
```

Ручное управление, когда Scaffold не подходит:

`Modifier.windowInsetsPadding(WindowInsets.safeDrawing)` — «отступи от всего, что может перекрыть контент».

И финальный гвоздь в «портрет всегда»: на больших экранах Android 16 (для targetSdk 36) **игнорирует** ограничения ориентации и запрет ресайза — приложение обязано переживать любые размеры окна. Проверяйте не «на моём телефоне», а resizable-эмулятором и split-screen.

Проверь себя.

- *Почему нельзя ветвиться по «это планшет»?* — Планшет может показать вас в узком окне split-screen, телефон в ландшафте — в широком: важна ширина окна, а не устройство.
- *Где живёт `selectedBenefitId` для двухпанельного режима?* — В том же едином STATE: раскладка сама решает, открывать экран или заполнять панель.
- *Клавиатура — это inset?* — Да: `imePadding()` приподнимает контент над ней так же, как `systemBars` — над панелями.

Тема 20. Interop с View как инженерная дисциплина, а не «вставил и молись»

Зачем это знать. Часть 1/2 честно ответила, где View/XML в 2026 остаётся нормой (камера, видео, WebView, карты, виджеты). Но она не ответила, как правильно встраивать View в Compose — а это как раз то место, где джун сажает утечки памяти, которые всплывают только в проде. Interop — это прежде всего вопрос: **кто владеет жизненным циклом чужого объекта.**

Аналогия. View внутри Compose — гость из другой страны. Ему нужны: переводчик (мост данных — `update`), правила заселения и выселения (`factory`/`onRelease`) и уважение к его распорядку (lifecycle: WebView и MapView живут по расписанию Activity, а не композиции).

20.1. `AndroidView`: четыре двери, у каждой своя роль

kotlin

```
AndroidView(  
    factory = { context ->  
        // Вызывается ОДИН раз на вход в композицию: создание + одноразовая на  
        LegacyChartView(context).apply {  
            onPointSelected = { id -> onAction(BenefitsAction.PointClicked(id))  
        }  
    },  
    update = { view ->  
        // Вызывается при КАЖДОЙ рекомпозиции, где изменились прочитанные тут s  
        // Только доставка данных. Не пересоздавать, не подписывать заново!  
        view.setPoints(state.chartPoints)  
    },  
    onReset = { view ->  
        // Только для lazy-контейнеров: View уходит в пул переиспользования — в  
        view.clear()  
    },  
    onRelease = { view ->  
        // View навсегда покидает композицию: отписки, остановка анимаций, осво  
        view.onPointSelected = null  
        view.stopAnimations()  
    },  
)
```

Два железных правила. **Первое:** тяжёлая инициализация и подписки — в `factory`, доставка свежих данных — в `update`; если вы создаёте объекты в `update`, вы создаёте их на каждую рекомпозицию. **Второе:** в `LazyColumn`/`Pager` используйте перегрузку с `onReset` — тогда Compose *переиспользует* дорогие View при скролле (как ViewHolder в RecyclerView), вместо создания новых; без `onReset` каждая карточка с View создаётся заново.

20.2. Гости с расписанием: WebView, MapView, PlayerView

У некоторых View есть собственный жизненный цикл, который обязан идти в ногу с Activity, иначе — утечки, «съеденная» батарея и краши при повороте. Классический мост:

kotlin

@Composable

```
fun rememberLifecycleAwareWebView(): WebView {
    val context = LocalContext.current
    val webView = remember { WebView(context) }
    val lifecycleOwner = LocalLifecycleOwner.current

    DisposableEffect(lifecycleOwner) {
        val observer = LifecycleEventObserver { _, event ->
            when (event) {
                Lifecycle.Event.ON_RESUME -> webView.onResume()
                Lifecycle.Event.ON_PAUSE -> webView.onPause()
                else -> Unit
            }
        }
        lifecycleOwner.lifecycle.addObserver(observer)
        onDispose {
            lifecycleOwner.lifecycle.removeObserver(observer)
            webView.destroy() // без этого — утечка процесса рендерера
        }
    }
    return webView
}
```

Для карт это же делает официальная Maps Compose library ([GoogleMap](#) composable сам пробрасывает lifecycle в MapView — см. errata №1 в конце), для видео — обёртки media3. Мораль: прежде чем писать мост руками, проверьте, нет ли официальной Compose-обёртки, которая уже решила lifecycle за вас.

20.3. Остальной чек-лист границы

- **XML-разметка целиком** — `AndroidViewBinding(MyLayoutBinding::inflate)`: для переходного периода и мелких legacy-кусков, не как стратегия. С `FragmentManagerView` внутри — отдельные пляски, избегайте.
- **Вложенный скролл** (Compose-список внутри View-скролла или наоборот): подключайте `Modifier.nestedScroll(rememberNestedScrollInteropConnection())`, иначе два мира дерутся за жест.
- **Фокус и доступность** через границу в целом работают, но порядок обхода TalkBack на гибридном экране — проверять руками: автоматика иногда «прыгает» между мирами.
- **Тесты гибридного экрана** — Espresso для View-части + `ComposeTestRule` для Compose-части в одном тесте (Тема 17.3).
- **Норма или долг? Interop** — норма, когда Compose-эквивалента нет физически (Surface-поверхности, WebView, RemoteViews-миры из части 1/2). Interop — долг, когда это ваш собственный старый custom view, у которого эквивалент есть: тогда рядом с ним должен лежать план миграции, а не оправдание.

Проверь себя.

- Почему подписку на listener делают в `factory`, а не в `update`? — `update` зовётся на каждую рекомпозицию: получите многократные подписки/пересоздания.
- Зачем `onReset` в `LazyColumn`? — Включает переиспользование View из пула при скролле вместо создания новых.
- Кто должен звать `webView.destroy()`? — Вы, в `onDispose`/`onRelease`: Compose не знает о внутренностях чужого View.

Тема 21. Перформанс как процесс: гипотеза → трейс → фикс → бенчмарк → страховка

Зачем это знать. Часть 1/2 дала модель (два вида «пропусков», конвейер кадра). Но модель без процесса — это интуиция, а интуиция в перформансе врёт постоянно. Senior отличается не знанием «десяти трюков оптимизации», а тем, что не делает ни одного фикса без измерения до и после.

Аналогия. Хороший врач не назначает лечение «на глазок»: анализы → диагноз → лечение → контрольные анализы. «Больной» кадр лечится так же, и у каждого шага есть свой инструмент.

21.1. Правило №0: debug-сборку не измеряем никогда

Debug-сборка медленнее релизной *по-другому в разных местах*: код не оптимизирован **R8** (оптимизатор и сжиматель кода при сборке release), не прогрев **AOT** (Ahead-Of-Time — заранее скомпилированный в машинный код; его антипод **JIT**, Just-In-Time — компиляция «на лету» по мере исполнения), включены отладочные проверки. Вывод, полученный на debug, не переносится на прод. Измеряют release-сборку с флагом `<profileable android:shell="true"/>` в манифесте (**profileable** — разрешение снимать профили производительности с релизной сборки без root).

21.2. Macrobenchmark: измерение как код

Macrobenchmark — отдельный тестовый модуль, который запускает *настоящее* приложение и меряет его снаружи, как пользователь:

```
kotlin

@Test
fun benefitsScrollJank() = benchmarkRule.measureRepeated(
    packageName = "com.bbk.app",
    metrics = listOf(FrameTimingMetric()),           // тайминги кадров: P50/P90
    iterations = 10,                                // одиночный прогон — не и
    startupMode = StartupMode.WARM,
    compilationMode = CompilationMode.Partial(),    // с Baseline Profile (см.
) {
    pressHome()
    startActivityAndWait()
    device.findObject(By.res("benefits_list")).fling(Direction.DOWN)
}
```

Словарь метрик. `StartupTimingMetric` меряет старт: **TTID** (Time To Initial Display — первый кадр на экране) и **TTFD** (Time To Full Display — приложение реально готово; о готовности вы сообщаете сами вызовом `reportFullyDrawn()`). `FrameTimingMetric` — длительности кадров по перцентилям (**P90** — «90% кадров быстрее этого значения»; хвосты P95/P99 и есть видимый jank) и `frameOverrunMs` — на сколько кадр промахнулся мимо дедлайна. `TraceSectionMetric` меряет ваши собственные участки: обернули подозрительный код в `trace("LoadBenefits") { ... }` — получили его длительность в отчёте. Виды старта: **cold** — процесс поднимается с нуля (самый честный и самый дорогой), **warm** — процесс жив, Activity создаётся заново, **hot** — Activity просто возвращается на передний план.

21.3. Baseline Profile: включить ≠ проверить

Baseline Profile — список «горячих» методов, которые компилируются в машинный код заранее, до первого запуска (иначе первые запуски работают через JIT и тормозят). Два обязательных пункта, которые все пропускают: (1) профиль надо **сгенерировать под свои экраны** (baselineprofile-плагин + сценарий прохода по критическим путям), а не надеяться на профили библиотек; (2) профиль надо **проверить**: тот же Macrobenchmark, два прогона — `CompilationMode.None` (без профиля) против `CompilationMode.Partial()` (с профилем). Разница на *ваших* метриках — единственное доказательство пользы. «Мы подключили Baseline Profiles» без такого сравнения — карго-культ.

21.4. Прод и трейсы: JankStats и Perfetto

Бенчмарки ловят регрессии до релиза; **JankStats** ловит их у реальных пользователей: колбэк на каждый кадр с `isJank` и — главное — **state tags**, привязывающие кадр к контексту:

```
kotlin

val holder = PerformanceMetricsState.getHolderForHierarchy(view)
holder.state?.putState("screen", "Benefits") // теперь в отчёте видно, НА КАКОМ
```

Когда метрики сказали «где болит», отвечает на вопрос «почему» **Perfetto** — системная запись всего, что делали потоки. Во вкладке FrameTimeline janky-кадры подсвечены, и видно виновника: main thread занят (рекомпозиция? GC — пауза сборщика мусора? ваш JSON-парсинг?), или RenderThread, или вообще система. Это тот самый инструмент из кейса части 1/2, где «main thread был занят верификацией классов».

Замыкает цикл **отчёт стабильности компилятора** — связь с Темой 11.3 части 2/2:

```
kotlin

// build.gradle.kts модуля
composeCompiler {
    metricsDestination = layout.buildDirectory.dir("compose_metrics")
    reportsDestination = layout.buildDirectory.dir("compose_reports")
}
```

В `*-classes.txt` видно, какие классы компилятор считает stable/unstable, в `*-composables.txt` — какие функции skipable. Нашли unstable `BenefitsState`? Вспоминаем: `ImmutableList` или `@Immutable` (Тема 11.3), потому что через границы модулей инференс не работает.

21.5. Дисциплина чисел

Один прогон — это не измерение, это анекдот. Минимум ~10 итераций; сравнивайте **медианы** и смотрите на хвосты (P90/P99), а не на среднее — среднее искажается выбросами (фоновая синхронизация, троттлинг). Фикс считается фиксом, когда разница между «до» и «после» больше разброса между прогонами «до». И финальный шаг цикла, о котором забывают: победивший бенчмарк остаётся в CI как **регрессионный страж** — иначе через три релиза оптимизацию молча съедят обратно.

Проверь себя.

- *Почему нельзя мерить debug?* — Нет R8/AOT и включены отладочные проверки: цифры не переносятся на прод.
 - *Чем TTID отличается от TTFD?* — Первый кадр против «реально готово к работе» (о втором вы сообщаете сами через `reportFullyDrawn()`).
 - *Как доказать пользу Baseline Profile?* — Macrobenchmark: `CompilationMode.None` против `Partial()` на своих сценариях, сравнение медиан.
-

Тема 22. Механика нетривиального UI: Modifier, Layout, жесты и анимации

Зачем это знать. Часть 1/2 описала рендеринг «сверху» (фазы, кадры, VSYNC). Но однажды дизайнер принесёт компонент, которого нет в Material, — и придётся спуститься «вниз»: свой Layout, свои жесты, свои анимации. Здесь собран минимум, без которого нетривиальный компонент превращается в набор случайных попыток.

22.1. Порядок Modifier'ов — это не стиль, а семантика

Модификаторы оборачивают друг друга сверху вниз: каждый следующий работает *внутри* предыдущего. Классика, на которой ловят на собеседовании:

```
kotlin
```

```
// Кликается ТОЛЬКО текст: padding снаружи clickable — «мёртвая» рамка без кликов
Text("Нажми", Modifier.padding(24.dp).clickable(onClick = onClick))

// Кликается ВСЯ область с отступом: clickable снаружи, padding внутри него
Text("Нажми", Modifier.clickable(onClick = onClick).padding(24.dp))
```

То же правило объясняет `size` до/после `padding`, порядок `background`/`clip` и т.д. Читайте цепочку как матрёшку: первый модификатор — самая внешняя кукла.

22.2. Constraints: переговоры родителя и ребёнка

Layout в Compose — короткие переговоры в три реплики: (1) родитель передаёт ребёнку **constraints** (диапазон «ширина от X до Y, высота от A до B»); (2) ребёнок сам выбирает свой размер внутри диапазона; (3) родитель расставляет детей по координатам. Жёсткое правило рантайма: **каждого ребёнка можно измерить только один раз за проход** — попытка перемерить бросит исключение. Это то самое правило одного прохода, которое делает layout Compose быстрым на глубоких деревьях. Легальный способ «узнать заранее» («какой ширины ты хотел бы быть?») — **intrinsic measurements** (внутренние измерения): `Modifier.width(IntrinsicSize.Max)` и им подобные — они делают отдельный «прикидочный» опрос, не нарушая правило.

Свой Layout — это одна функция с теми же тремя репликами:

```
kotlin

@Composable
fun SimpleColumn(modifier: Modifier = Modifier, content: @Composable () -> Unit) {
    Layout(content = content, modifier = modifier) { measurables, constraints ->
        // (1)+(2): меряем каждого ребёнка РОВНО ОДИН РАЗ
        val placeables = measurables.map { it.measure(constraints) }
        val height = placeables.sumOf { it.height }

        // (3): объявляем свой размер и расставляем детей
        layout(width = constraints.maxWidth, height = height) {
            var y = 0
            placeables.forEach { placeable ->
                placeable.placeRelative(x = 0, y = y)
                y += placeable.height
            }
        }
    }
}
```

`placeRelative` (а не `place`) — бесплатная поддержка RTL (right-to-left — языки с письмом справа налево: арабский, иврит): координаты зеркалятся автоматически.

22.3. `graphicsLayer`: дёшево двигать, вращать, растворять

Смещение, поворот, масштаб и прозрачность через `Modifier.graphicsLayer { translationX = offset; alpha = a }` применяются на фазе **Drawing** — без повторных Composition и Layout. Лямбда-версия ещё и откладывает чтение состояния до фазы рисования (deferred read из перф-плейбука части 1/2): анимация в 60 кадров дёргает только отрисовку, а не всё дерево. Правило: анимируете геометрию появления — `Modifier.offset {}`/`graphicsLayer {}`; меняете структуру — тогда уже честная рекомпозиция.

22.4. Lazy-списки: `key` и `contentType`

kotlin

```
LazyColumn {
    items(
        items = state.items,
        key = { benefit -> benefit.id },          // стабильная ИДЕНТИЧНОСТЬ элем
        contentType = { benefit -> benefit.kind }, // «порода» элемента для пул
    ) { benefit ->
        BenefitCard(benefit, Modifier.animateItem()) // анимация перестановок —
    }
}
```

`key` отвечает на вопрос «это тот же элемент, что и раньше?»: без него состояние элемента (например, раскрытый аккордеон из Темы 14) приклеено к *позиции* и при вставке сверху «переезжает» на чужой элемент; с ним — следует за элементом, а `animateItem()` красиво анимирует перестановки. `contentType` — подсказка пулу переиспользования: карточки и заголовки секций не будут претендовать на одни и те же заготовки. Оба параметра — прямые родственники ViewHolder-паттерна, только декларативные.

22.5. Жесты + `Animatable`: палец всегда важнее анимации

Отличие живого UI от «дёрганого» — прерываемость: если анимация возврата ещё летит, а пользователь снова схватил элемент, элемент должен подчиниться пальцу мгновенно. Для этого анимируемое значение держат в `Animatable`:

kotlin

```
val scope = rememberCoroutineScope()
val offsetX = remember { Animatable(0f) }

Box(
    modifier = Modifier
        .graphicsLayer { translationX = offsetX.value } // чтение отложено до D
        .pointerInput(Unit) {
            detectDragGestures(
                onDrag = { _, drag ->
                    scope.launch { offsetX.snapTo(offsetX.value + drag.x) } //
                },
                onDragEnd = {
                    scope.launch { offsetX.animateTo(0f, spring()) } // отпусти
                },
            )
        },
) { /* контент */ }
```

`snapTo` телепортирует значение, `animateTo` едет плавно — и любой новый `snapTo`/`animateTo` **отменяет** предыдущую анимацию: прерываемость встроена. Фокус и клавиатура для полноты картины: программный фокус — `FocusRequester` + `Modifier.focusRequester()` + `requestFocus()` в `LaunchedEffect`; поведение кнопки Enter на клавиатуре — `KeyboardOptions(imeAction = ...)` + `KeyboardActions`.

22.6. Шпаргалка выбора анимационного API

Задача	Инструмент
Показать/скрыть блок	<code>AnimatedVisibility</code>
Плавно сменить контент (счётчик, вкладка)	<code>AnimatedContent</code> / <code>Crossfade</code>
Одно значение следует за state	<code>animateDpAsState</code> , <code>animateColorAsState</code> , ...
Несколько значений синхронно по одному state	<code>updateTransition</code>
Жесты, прерываемость, ручное управление	<code>Animatable</code>
Вечная анимация (шиммер, пульс)	<code>rememberInfiniteTransition</code>
Перестановки в списке	<code>Modifier.animateItem()</code> при заданных <code>key</code>

Проверь себя.

- Почему `.padding().clickable()` и `.clickable().padding()` кликаются по-разному? — Модификаторы оборачивают друг друга: кликабельно только то, что внутри `clickable`.
- Что случится, если измерить ребёнка дважды? — Исключение рантайма: правило одного прохода; «спросить заранее» — только через `intrinsic`s.
- Почему для драга берут `Animatable`, а не `animateFloatAsState`? — Нужны `snapTo` за пальцем и прерываемость анимации новым жестом.

Правки и уточнения к частям 1/2 и 2/2 (errata)

Хороший конспект честно исправляет сам себя. Семь правок — от фактических до терминологических.

Правка 1 (часть 1/2). Карты — не «зона без Compose-ответа». Google предоставляет официальную **Maps Compose library**: `GoogleMap`-composable, маркеры, полигоны, управление камерой как state. Внутри работает тот же Maps SDK (и его `renderer/lifecycle/` ограничения наследуются), но на уровне публичного UI API Compose-ответ существует. Корректная формулировка: *«Карты имеют официальные Compose-обёртки, но наследуют lifecycle и performance-ограничения базового Maps SDK»* — в отличие от `WebView` и кастомных уведомлений, где обёртки нет вовсе.

Правка 2 (часть 1/2). «У Compose нет собственного эквивалента выделенной Surface» — слишком категорично. В Compose Foundation есть

`AndroidExternalSurface` (выделенная Surface отдельным слоем, по умолчанию позади окна) и `AndroidEmbeddedExternalSurface` (Surface, встроенная в иерархию UI). Точная формулировка: *«Compose не заменяет платформенную модель Surface/SurfaceFlinger, но предоставляет собственные API для владения и размещения dedicated Surface»*.

Вывод части 1/2 («камера/видео/GL живут на Surface-поверхностях») остаётся верным — неточна была только категоричность.

Правка 3 (часть 2/2, Тема 8). Логирование дропов в `Channel (DROP_OLDEST)` было нерабочим. В нашем примере стояло `if (result.isFailure) { Timber... }` — но при `DROP_OLDEST` `trySend` практически всегда *успешен*: канал молча вытесняет старейший элемент, и `isFailure` главный сценарий потери не ловит. Рабочая наблюдаемость — параметр `onUndeliveredElement`:

kotlin

```
private val newsChannel: Channel<NEWS> = Channel(
    capacity = NEWS_BUFFER_CAPACITY,
    onBufferOverflow = BufferOverflow.DROP_OLDEST,
    onUndeliveredElement = { droppedNews ->
        // Сюда попадает и вытесненный при переполнении элемент, и неполученный
        Timber.tag(TAG).e("mylog 003: News dropped: $droppedNews")
    },
)
```

Нюансы: колбэк вызывается из произвольного контекста — внутри только лог/метрика, никаких исключений и тяжёлой работы.

Правка 4 (часть 2/2, Темы 8–9). `state + acknowledgment` — не `exactly-once`.

Контрпример: навигация выполнена → процесс умер до отправки `NavigationHandled` → после восстановления `pendingNavigation` всё ещё в сохранённом состоянии → переход повторится. Точное название гарантии: **durable intent + дедупликация**, что даёт *effectively-once* при **идемпотентном** потребителе (идемпотентность — свойство «повторное выполнение не меняет результат»: повторно открыть уже открытый экран — безвредно). Для внешних *необратимых* действий (платёж, интент в чужое приложение) `exactly-once` недостижим без транзакции, объединяющей действие и подтверждение, — поэтому такие действия делают идемпотентными на стороне получателя (`idempotency key`).

Правка 5 (часть 2/2, Тема 7). Уточнение про проглоченный

`CancellationException`. Формулировка «скоуп продолжит считать корутину живой» неточна. Точная механика: если поймать `CancellationException` и не пробросить, код после `catch` продолжит выполняться, и корутина завершится как *нормально завершившаяся*. Проблема не в «вечно живой» корутине, а в том, что **отмена превращается в обычный control flow**: корутина игнорирует запрос на остановку и продолжает работу (сеть, запись в базу), нарушая контракт `structured concurrency` — «отменённая корутина завершается быстро и именно отменой». Дополнительно: родительский `scope` не считает `CancellationException` ошибкой — поэтому его нельзя «репортить как краш», его нужно пробрасывать. Практическое правило части 2/2 (**rethrow всегда**) остаётся в силе — меняется только объяснение «почему».

Правка 6 (часть 2/2, Тема 4). Версию MVICore надо оговаривать в начале, а не в конце. Правильная рамка для читателя: «Ниже разбирается семантика *MVICore 1.x* на *RxJava 2* — версии, используемой проектом. Публичный *MVICore 2.0.0* — это перевод на *RxJava 3*: модель компонентов (*Feature/Actor/Reducer/News*) не меняется, меняется *runtime-зависимость*». Следствие для стратегии из части 2/2: миграция 1.x → 2.x сама по себе даёт мало, если цель — `coroutine-native`; это замена Rx2 на Rx3, а не уход от Rx.

Правка 7 (часть 1/2). Источники должны быть проверяемыми. Пометки вида «Android Developers + 6» — след автогенерации: по ним нельзя проверить конкретное утверждение. Стандарт оформления для следующих версий: у каждого числа/версии/сильного тезиса — конкретный источник с названием страницы, датой доступа и типом (`official docs` / `API reference` / `source code` / `case study` / `secondary`). Раздел «Куда смотреть» ниже оформлен по этому стандарту.

Мини-словарь части 3/3

- **Snapshot** — согласованный «снимок» всех state-ячеек на момент времени; механизм, которым Compose отслеживает чтения и записи.
- **Инвалидация** — пометка «это место устарело, перерисуй при следующем кадре».
- **Сериализация** — превращение объекта в байты/примитивы для сохранения или передачи.
- **Bundle** — системный контейнер «ключ → значение» для сохранения мелких данных; лимит порядка 1 МБ на транзакцию.
- **Process death** — система убила процесс приложения в фоне; при возврате всё пересоздаётся, но сохранённый Bundle возвращается.
- **IME (Input Method Editor)** — экранная клавиатура.
- **Composition (в контексте IME)** — «недособранный» текст, который клавиатура ещё уточняет (подчёркнутый при свайп-наборе).
- **TalkBack** — встроенный скринридер Android: озвучивает экран и даёт управление жестами.
- **Switch Access** — управление устройством одной-двумя физическими кнопками (моторные ограничения).
- **Semantics / merged tree** — машинно-читаемое описание экрана; merged-дерево сливает потомков интерактивных узлов в один узел.
- **testTag** — метка узла только для тестов.
- **Idle** — состояние «всё устаканилось»: нет незавершённых рекомпозиций/анимаций; тесты ждут его автоматически.
- **Back stack** — стопка открытых экранов; «назад» снимает верхний.
- **Deep link** — внешняя ссылка, открывающая конкретный экран приложения.
- **Predictive back** — жест «назад» с анимацией-предпросмотром места назначения (Android 14+).
- **Insets** — зоны, занятые системой: статус-бар, жестовая панель, вырез, клавиатура.
- **Cutout** — вырез в экране под камеру.
- **Edge-to-edge** — приложение рисует под системными панелями, от края до края (принудительно с targetSdk 35).
- **Window size class** — класс ширины окна (Compact/Medium/Expanded + новые Large/XL); основа adaptive-раскладок.
- **Fold posture** — поза складного устройства: раскрыт, «книжка», «палатка» (tabletop).
- **AOT / JIT** — компиляция заранее / «на лету» во время работы.
- **R8** — оптимизатор, сжиматель и обфускатор кода release-сборки.
- **Profileable** — флаг манифеста, разрешающий снимать профили производительности с release-сборки.
- **TTID / TTFD** — время до первого кадра / до полной готовности (`reportFullyDrawn()`).
- **Перцентиль (P90)** — «90% измерений быстрее этого значения»; хвосты P95/P99 —

видимый jank.

- **GC** (Garbage Collector) — сборщик мусора; его паузы — частая не-Compose причина janky-кадров.
- **Идемпотентность** — повторное выполнение операции не меняет результат.
- **Baseline Profile** — список «горячих» методов для компиляции в машинный код до первого запуска.

Куда смотреть в официальной документации

- State hoisting и владельцы состояния — developer.android.com/develop/ui/compose/state-hoisting (*official docs*)
 - State-based text fields и миграция — developer.android.com/develop/ui/compose/text/migrate-state-based (*official docs*)
 - Semantics в Compose — developer.android.com/develop/ui/compose/semantics (*official docs*)
 - Тестирование Compose-layout — developer.android.com/develop/ui/compose/testing (*official docs*)
 - Navigation 3 — developer.android.com/guide/navigation/navigation-3 (*official docs*)
 - Predictive back — developer.android.com/develop/ui/compose/system/predictive-back (*official docs*)
 - Material 3 Adaptive (релизы) — developer.android.com/jetpack/androidx/releases/compose-material3-adaptive (*release notes*)
 - Insets в Compose — developer.android.com/develop/ui/compose/layouts/insets (*official docs*)
 - Views в Compose (interop, onReset/onRelease) — developer.android.com/develop/ui/compose/migrate/interoperability-apis/views-in-compose (*official docs*)
 - Baseline Profiles — developer.android.com/develop/ui/compose/performance/baseline-profiles (*official docs*)
 - Macrobenchmark — developer.android.com/topic/performance/benchmarking/macrobenchmark-overview (*official docs*)
 - Intrinsic measurements — developer.android.com/develop/ui/compose/layouts/intrinsic-measurements (*official docs*)
 - Maps Compose library — developers.google.com/maps/documentation/android-sdk/maps-compose (*official docs*)
 - `Channel` и `onUndeliveredElement` — kotlinlang.org/api/kotlinx.coroutines (страница Channel) (*API reference*)
 - Релизы MVICore — github.com/badoo/MVICore/releases (*source/releases*)
-

Серия закрыта: 5.8 — рендеринг и границы Compose, 5.9 — MVI-архитектура под микроскопом, 5.10 — инженерия самого Compose. Три конспекта вместе покрывают и «как устроено», и «как спроектировано», и «как это писать, тестировать и не сломать».

Удачи — теперь у вас есть система, а не набор фактов. 🚀